

[← Back to blog](#)

[~/blog](#)

Building a SQL Agent with MLflow Observability

May 31, 2026 • 9 min read • By Mohammed Vasim



[sql](#) [mlflow](#) [langchain](#) [agents](#) [observability](#)

On this page



What We're Building

Ever wanted to see inside your AI agent's head? Most agent tutorials show you the happy path but leave you guessing when something breaks. This notebook is different — we're building a SQL agent that queries the Chinook sample database, and we're instrumenting every step with MLflow tracing so we can actually see what's happening.

The stack:

- **NVIDIA's API** via LangChain for the LLM (gpt-oss-120b)
- **MLflow** for tracing every tool call the agent makes
- **Chinook** — the classic SQLite sample music store database

By the end, you'll have an agent that can answer questions like "which genre on average has the longest tracks?" and you'll be able to inspect every query it ran to get there.

Project Dependencies

Here's the full list of dependencies we need. I'm using uv/pip with pinned versions for reproducibility:

- **langchain** — The orchestration framework
- **langchain-nvidia-ai-endpoints** and **langchain-openai** — Model providers (we'll use NVIDIA)
- **langgraph** — For building agent graphs
- **mlflow** — The tracing and observability layer
- **ipykernel** — To run this as a notebook

pyproject.toml :

toml

Copy

```
[
project
]
name = "sql-agent-mflow"
version = "0.1.0"
requires-python = "≥ 3.13"
dependencies = [
    "ipykernel ≥ 7.2.0",
    "langchain ≥ 1.3.2",
    "langchain-nvidia-ai-endpoints ≥ 1.4.0",
    "langchain-openai ≥ 1.2.2",
    "langgraph ≥ 1.2.2",
    "mlflow ≥ 3.12.0",
]
```

Setting Up the Environment

I'm using NVIDIA's API instead of OpenAI directly — it's cheaper for development and gives access to solid open-source models. The OpenAI client library is compatible, we just change the base URL.

python

Copy

```
import os
from langchain.chat_models import init_chat_model
import mlflow

API_KEY = os.environ["NVIDIA_API_KEY"]
BASE_URL = "https://integrate.api.nvidia.com/v1"
CHAT_MODEL_NAME = "openai/gpt-oss-120b"
```

Initializing the Chat Model

LangChain's `init_chat_model` is a unified way to create any chat model. We just tell it which model, which provider, and where the API lives. Under the hood it creates the right client for NVIDIA's endpoints.

python

 Copy

```
llm = init_chat_model(CHAT_MODEL_NAME, model_provider="nvidia", base_url=BASE_URL)
```

Quick Smoke Test

Always check the connection before building on top of it. A simple ping/pong confirms the model is reachable and credentials are correct.

python

 Copy

```
llm.invoke("Ping")
```

Setting Up MLflow Tracing

This is the observability layer. MLflow captures every LLM call, tool invocation, and agent step as traces that I can inspect. It's like having a debugger that records everything.

I'm running an MLflow server locally on port 5909. The tracking URI tells MLflow where to send the trace data, and the experiment groups all traces under one project.

python

Copy

```
TRACKING_URI = "http://localhost:5909"
mlflow.set_tracking_uri(TRACKING_URI)
mlflow.set_experiment("sql-agent")
```

Adding Traces with @mlflow.trace

The `@mlflow.trace` decorator is the simplest way to instrument a function. Every time this function is called, MLflow records the input, output, and any metadata we attach. I started with a version that tracks user and session IDs for multi-user scenarios.

python

Copy

```
@mlflow.trace
def ask_ai(query: str, user_id: str, session_id: str):
    """Ask AI any query"""
    mlflow.update_current_trace(
        metadata={
            "mlflow.trace.user": user_id,
            "mlflow.trace.session": session_id
        }
    )
    return llm.invoke("Ping")
```

Let's test it. The function should return a response, and MLflow should record the trace.

python

Copy

```
print(ask_ai("Hello", "123", "456"))
```

For this demo, the user/session metadata adds complexity without much value. Let me simplify to a cleaner version that just passes the query through.

python

Copy

```
@mlflow.trace
def ask_ai(query: str):
    """Ask AI any query"""
    return llm.invoke(query)
```

Quick test to confirm the simplified version works.

```
python
```

[Copy](#)

```
ask_ai("Hi")
```

Enabling LangChain Autolog

MLflow can automatically instrument all LangChain calls without any decorators. This means every LLM call, tool, chain, and agent step gets traced automatically. It's the easiest path to full observability.

```
python
```

[Copy](#)

```
mlflow.langchain.autolog()
```

Getting the Chinook Database

Chinook is a sample SQLite database that models a music store. It has artists, albums, tracks, customers, invoices — a realistic dataset that's great for testing SQL agents.

Google hosts a copy on Cloud Storage. Let's download it.

```
python
```

[Copy](#)

```
import requests, pathlib

url = "https://storage.googleapis.com/benchmarks-artifacts/chinook/Chinook.db"
local_path = pathlib.Path("Chinook.db")

if local_path.exists():
```

```
print(f"{local_path} already exists, skipping download.")
else:
    response = requests.get(url)
    if response.status_code == 200:
        local_path.write_bytes(response.content)
        print(f"File downloaded and saved as {local_path}")
    else:
        print(f"Failed to download the file. Status code: {response.status_code}")
```

Exploring the Schema

Before building an agent, I want to know what we're working with. Let's list the tables and grab a sample from one of them. This also confirms the database downloaded correctly.

python

 Copy

```
import sqlite3

con = sqlite3.connect("Chinook.db")
cursor = con.cursor()

cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
tables = [row[0] for row in cursor.fetchall() if not row[0].startswith("sqlite")]

print("Dialect: sqlite")
print(f"Available tables: {tables}")

cursor.execute("SELECT * FROM Artist LIMIT 5;")
print(f"Sample output: {cursor.fetchall()}")
con.close()
```

Building SQL Tools for the Agent

Now for the core of the agent — the SQL tools. I'm creating four tools that let the agent interact with the database:

1. **sql_db_list_tables** — Lists all tables (agent calls this first to understand the schema)
2. **sql_db_schema** — Gets the CREATE TABLE statement + sample rows for a given

table

3. **sql_db_query** — Executes a SQL query and returns results
4. **sql_db_query_checker** — Reviews a query for common mistakes before execution

The checker tool is especially useful — it uses the LLM itself to catch issues like `NOT IN` with NULL values or wrong JOIN columns before the query hits the database.

Let's define them one at a time, starting with the simplest.

python

Copy

```
import sqlite3
from langchain.tools import tool

# Below are minimal tools for demonstration purposes.
# They are not intended to be secure or for production use.

@tool
def sql_db_list_tables() -> str:
    """Input is an empty string, output is a comma-separated list of tables in the database"""
    con = sqlite3.connect("Chinook.db")
    try:
        cursor = con.cursor()
        cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
        tables = [row[0] for row in cursor.fetchall() if not row[0].startswith('sqlite_')]
        return ", ".join(tables)
    finally:
        con.close()
```

The schema tool takes a list of table names and returns their CREATE TABLE statements plus sample rows. It validates that each table actually exists before querying.

python

Copy

```
@tool
def sql_db_schema(table_names: str) -> str:
    """Input to this tool is a comma-separated list of tables, output is the CREATE TABLE statements plus sample rows. Be sure that the tables actually exist by calling sql_db_list_tables first"""
```

```

Example Input: table1, table2, table3"""
con = sqlite3.connect("Chinook.db")
try:
    cursor = con.cursor()
    cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
    valid_tables = {row[0] for row in cursor.fetchall() if not row[0].startswith('sqlite_')}
    results = []
    for table in table_names.split(","):
        table = table.strip()
        if table not in valid_tables:
            results.append(f"Error: table_names {{{table!r}}} not found in database")
            continue
        cursor.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name='{table}';")
        schema_row = cursor.fetchone()
        if schema_row:
            results.append(schema_row[0])
        try:
            quoted_table = "'" + table.replace("'", "''") + "'"
            cursor.execute(f"SELECT * FROM {quoted_table} LIMIT 3;")
            rows = cursor.fetchall()
            if rows:
                col_names = [description[0] for description in cursor.description]
                results.append(f"/*\n3 rows from {table} table:\n" +
                               "\n".join("  " + col + ": " + str(row[col]) for col, row in enumerate(rows)))
        except Exception as e:
            results.append(f"Error fetching sample rows: {e}")
    return "\n\n".join(results)
finally:
    con.close()

```

The query tool executes a SQL statement and returns the raw results. If the query fails, it returns the error message so the agent can retry with a corrected query.

python

 Copy

@tool

```

def sql_db_query(query: str) -> str:
    """Input to this tool is a detailed and correct SQL query, output is a raw result.
    If the query is not correct, an error message will be returned.
    If an error is returned, rewrite the query, check the query, and try again.
    If you encounter an issue with Unknown column 'xxxx' in 'field list', use
    con = sqlite3.connect("Chinook.db")
    try:
        cursor = con.cursor()
        cursor.execute(query)
        res = cursor.fetchall()
    """

```



```

        return str(res)
    except Exception as e:
        return f"Error: {e}"
    finally:
        con.close()

```

The checker tool is the secret sauce. Before executing a query, the agent submits it to the LLM for review. This catches common SQL mistakes — NULL handling, UNION vs UNION ALL, BETWEEN edge cases — before they hit the database.

python

Copy

```

@tool
def sql_db_query_checker(query: str) -> str:
    """Use this tool to double check if your query is correct before executing.
    Always use this tool before executing a query with sql_db_query!"""
    trigger_prompt = """{query}
Double check the sqlite query above for common mistakes, including:
- Using NOT IN with NULL values
- Using UNION when UNION ALL should have been used
- Using BETWEEN for exclusive ranges
- Data type mismatch in predicates
- Properly quoting identifiers
- Using the correct number of arguments for functions
- Casting to the correct data type
- Using the proper columns for joins

If there are any of the above mistakes, rewrite the query. If there are no mistakes,
output the final SQL query only.

SQL Query: """.format(query=query)

    response = llm.invoke(trigger_prompt)
    return response.text.strip()

```

Now let's bundle them together so the agent can use all four.

python

Copy

```

tools = [sql_db_list_tables, sql_db_schema, sql_db_query, sql_db_query_checker]

```

Designing the System Prompt

The system prompt is the agent's personality. It needs to be very explicit about the workflow:

1. First, list the tables
2. Then, check the schema of relevant tables
3. Formulate a query and double-check it
4. Execute and return results

I also forbid DML statements — this is a read-only agent. The prompt templates in `{dialect}` and `{top_k}` so it can be reused for different databases.

python

Copy

```
system_prompt = """
You are an agent designed to interact with a SQL database.
Given an input question, create a syntactically correct {dialect} query to run
then look at the results of the query and return the answer. Unless the user
specifies a specific number of examples they wish to obtain, always limit your
query to at most {top_k} results.

You can order the results by a relevant column to return the most interesting
examples in the database. Never query for all the columns from a specific table
only ask for the relevant columns given the question.

You MUST double check your query before executing it. If you get an error while
executing a query, rewrite the query and try again.

DO NOT make any DML statements (INSERT, UPDATE, DELETE, DROP etc.) to the
database.

To start you should ALWAYS look at the tables in the database to see what you
can query. Do NOT skip this step.

Then you should query the schema of the most relevant tables.
""".format(
    dialect="sqlite",
    top_k=5,
)
```

Creating the Agent

LangChain's `create_agent` is the high-level way to bind a model, tools, and system prompt together. The agent will autonomously decide which tools to call and in what order, using the LLM's reasoning to plan its actions.

python

 Copy

```
from langchain.agents import create_agent

agent = create_agent(
    llm,
    tools,
    system_prompt=system_prompt,
)
```

Testing the Agent

Let's ask the agent a real question. I'm using `stream` mode so we can see each step as the agent plans and executes. For each query, the agent will:

1. List tables to understand the schema
2. Check relevant table schemas
3. Formulate and double-check a query
4. Execute it and format the response

All of this will be traced by MLflow — we can inspect every tool call, token usage, and latency.

python

 Copy

```
question = "Which genre on average has the longest tracks?"

for step in agent.stream(
    {"messages": [{"role": "user", "content": question}]},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

It also handles broader questions like data summaries well — asking for a “summary of data” triggers the agent to iterate through every table and compile aggregate statistics.

python

 Copy

```
question = "give summary of data"

for step in agent.stream(
    {"messages": [{"role": "user", "content": question}]},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

What We Built

A SQL agent with full observability. The agent can explore the Chinook database schema, formulate queries, double-check them for bugs, and return answers — all while MLflow records every step.

The coolest part is the `sql_db_query_checker` tool — it uses the LLM itself to review queries before execution. This catches common mistakes like `NOT IN` with NULL values or missing join conditions, making the agent more reliable than a naive approach.

If you want to take this further, you could:

- Connect multiple databases and let the agent choose which to query
- Add a retry mechanism for error recovery
- Visualize MLflow traces in a dashboard for debugging production agents
- Extend the tools to support UPDATE/INSERT with user confirmation

Comments (0)

No comments yet. Be the first to comment!

Leave a comment

Name

Your name

Email

you@example.com

Comment

Your comment...

Post Comment